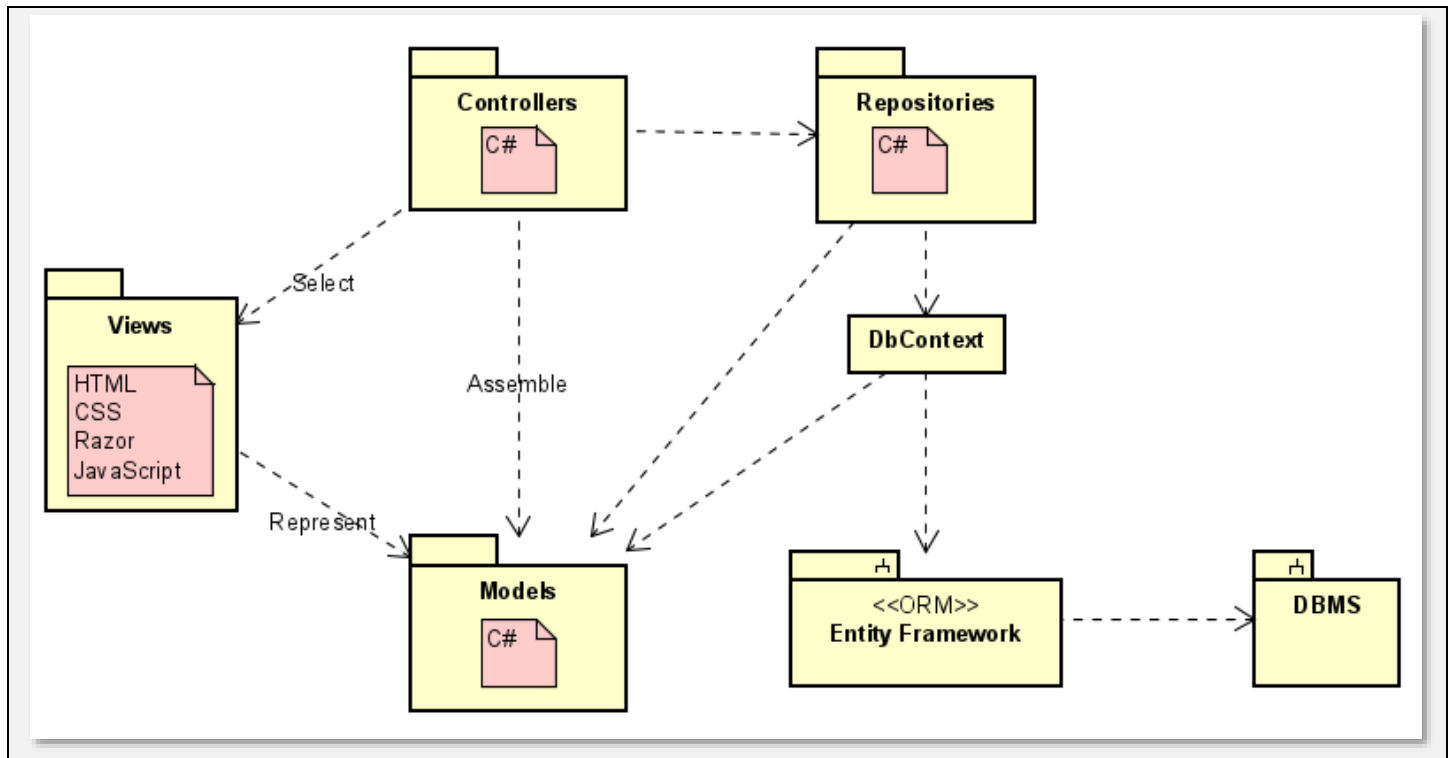
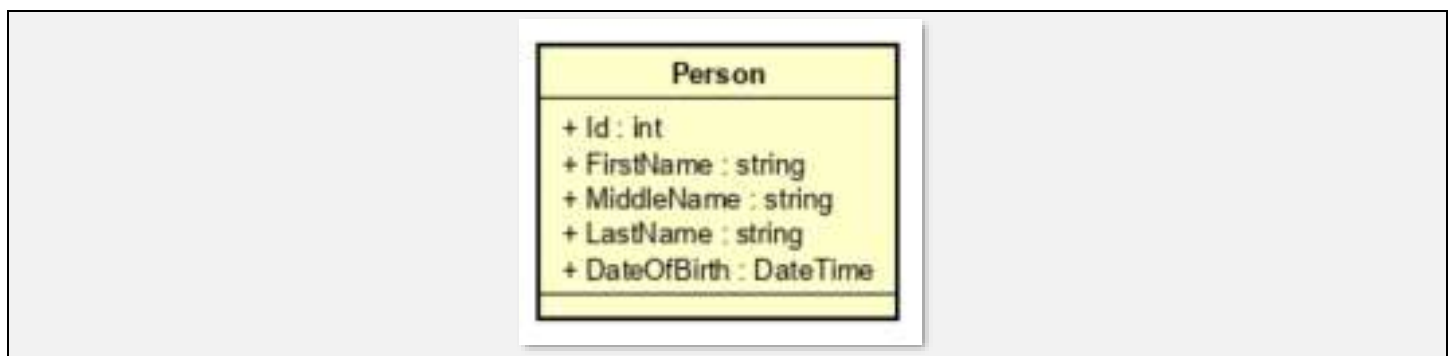


Architectural Overview



What is a Model?

- ❖ A model is an abstraction of the objects from the problem domain.
- ❖ It stores the data elements and how to process them.
- ❖ For example, let us assume that our problem domain involves tracking people.
 - Using the UML, the model for this domain is a **Person** class:



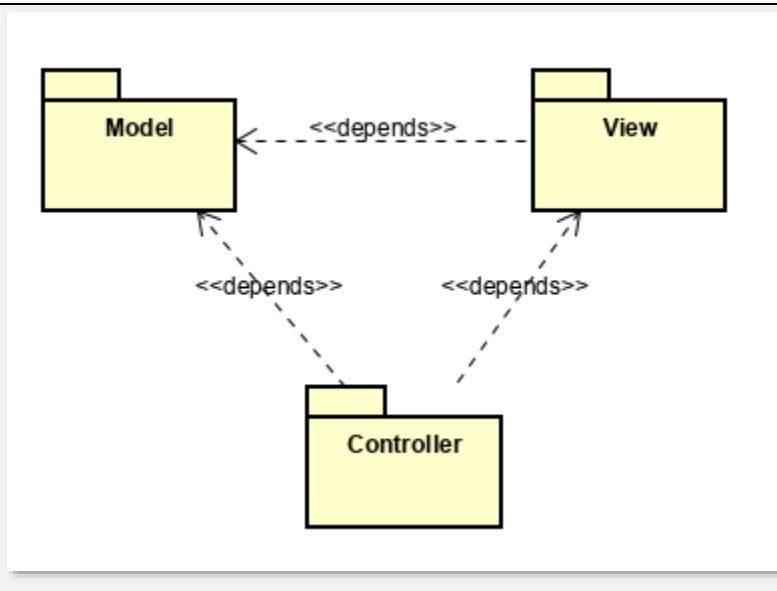
- ❖ Using C#, the model is then:

```

public class Person
{
    public int Id { get; set; }
}
  
```

```
public string FirstName { get; set; } = String.Empty;  
public string? MiddleName { get; set; }  
public string LastName { get; set; } = String.Empty;  
public DateTime DateOfBirth { get; set; }  
}
```

ASP.NET MVC Dependencies



- ❖ In the ASP.NET MVC architecture, the model does not depend on the view or the controller.
- ❖ However, the view and the controller depend on the model.
- ❖ The controller assembles the model and sends it to the appropriate view.
- ❖ The view's responsibility is to present a visual representation of the model.

Working with a Database

Object-Relational Mappers (ORM)

- ❖ A tool that connects object-oriented programming languages to relational databases.
- ❖ Converts database tables and rows into objects you can work with in your code.
- ❖ Works with major database systems (Oracle, DB2, Sybase, etc.)
- ❖ Lets you interact with databases using familiar object-oriented syntax instead of writing SQL.
- ❖ Uses similar interfaces to object databases, making it easier to learn.

Example of Object-Relational Mappers:

- ❖ Entity Framework (.NET)
- ❖ Hibernate (Java)
- ❖ Laravel (PHP)
- ❖ Django (Python)

Entity Framework

- ❖ Microsoft's ORM specifically designed for .NET applications.
- ❖ Target users: .NET developers (C#, VB.NET, F#, etc.)
- ❖ Main benefit: Work with database data as regular objects instead of raw database records.
- ❖ Code reduction: Eliminates most manual data-access code you'd normally have to write
- ❖ Domain-specific objects: Creates objects that match your business logic (like Customer, Order, Product)
- ❖ Real-world impact: Instead of writing SQL connection code, you can directly work with objects like `customer.Orders.Add(newOrder)`
- ❖ Microsoft ecosystem: Built-in integration with other Microsoft development tools and frameworks

Approaches

Code First Approach

- ❖ The code first approach involves creating the classes and then allowing the ORM to automatically create the database tables from the code.

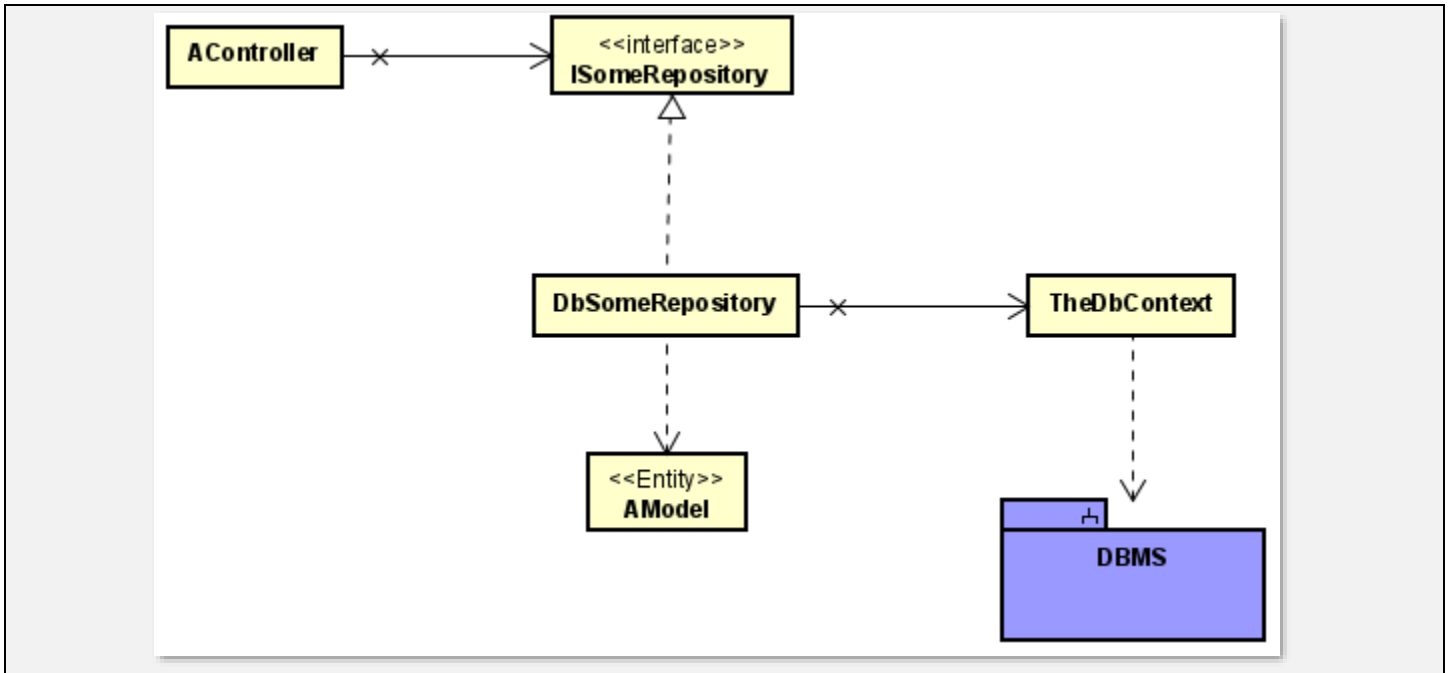
Schema or Model First Approach

- ❖ In the model first approach, the model of the database is created using a tool that creates diagrams like ERDs.
- ❖ The ORM then generates the classes and tables from the model.

Database First Approach

- ❖ In the database first approach, the database is created and then the ORM generates the model and the classes from the database.

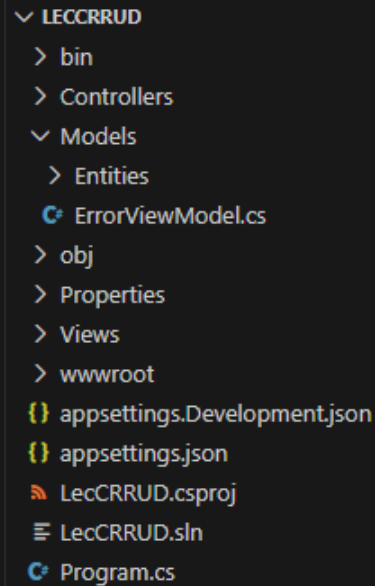
This Course's Class Pattern Structure



- ❖ The repository is configured as a service and is injected into the controllers through Dependency Injection.
- ❖ The repository accesses the database through the database context (i.e., the **DbContext**) to build the model.

Entities

- ❖ Entities are classes that map to a table in the database.
- ❖ **Recommendation:** Create a folder called *Entities* in the *Models* folder to house your entities



```
▼ LECCRRUD
  > bin
  > Controllers
  ▼ Models
    > Entities
    ErrorViewModel.cs
  > obj
  > Properties
  > Views
  > wwwroot
  {} appsettings.Development.json
  {} appsettings.json
  LecCRRUD.csproj
  LecCRRUD.sln
  Program.cs
```

E.g., The Person Entity

```
using System.ComponentModel.DataAnnotations;

namespace LecCRRUD.Models.Entities;

public class Person
{
    public int Id { get; set; }
    public string FirstName { get; set; } = String.Empty;
    public string? MiddleName { get; set; }
    public string LastName { get; set; } = String.Empty;
    [DataType(DataType.Date)]
    public DateTime DateOfBirth { get; set; }
}
```

- ❖ Use properties for the data elements.
- ❖ We sometimes must help the ORM by specifying more information about the data.
 - e.g., the `[DataType(DataType.Date)]` data annotation

Database Context Class

- ❖ The database context class is the bridge into the actual database.
- ❖ You interact with the database through this class.
- ❖ It must be derived from 'DbContext' from the 'Microsoft.EntityFrameworkCore' namespace.
- ❖ If the **Microsoft.EntityFrameworkCore** and the **Microsoft.EntityFrameworkCore.Sqlite** packages were not installed, use the **CLI** to install the following packages:
 - `dotnet add package Microsoft.EntityFrameworkCore.Sqlite`
 - `dotnet add package Microsoft.EntityFrameworkCore.Tools`

- dotnet add package Microsoft.EntityFrameworkCore.Design
- ❖ Note: add --version <num>.* to install the latest of a specific version
 - E.g. dotnet add package Microsoft.EntityFrameworkCore.Sqlite --version 9.*

```
using LecCRRUD.Models.Entities;
using Microsoft.EntityFrameworkCore;

namespace LecCRRUD.Services;

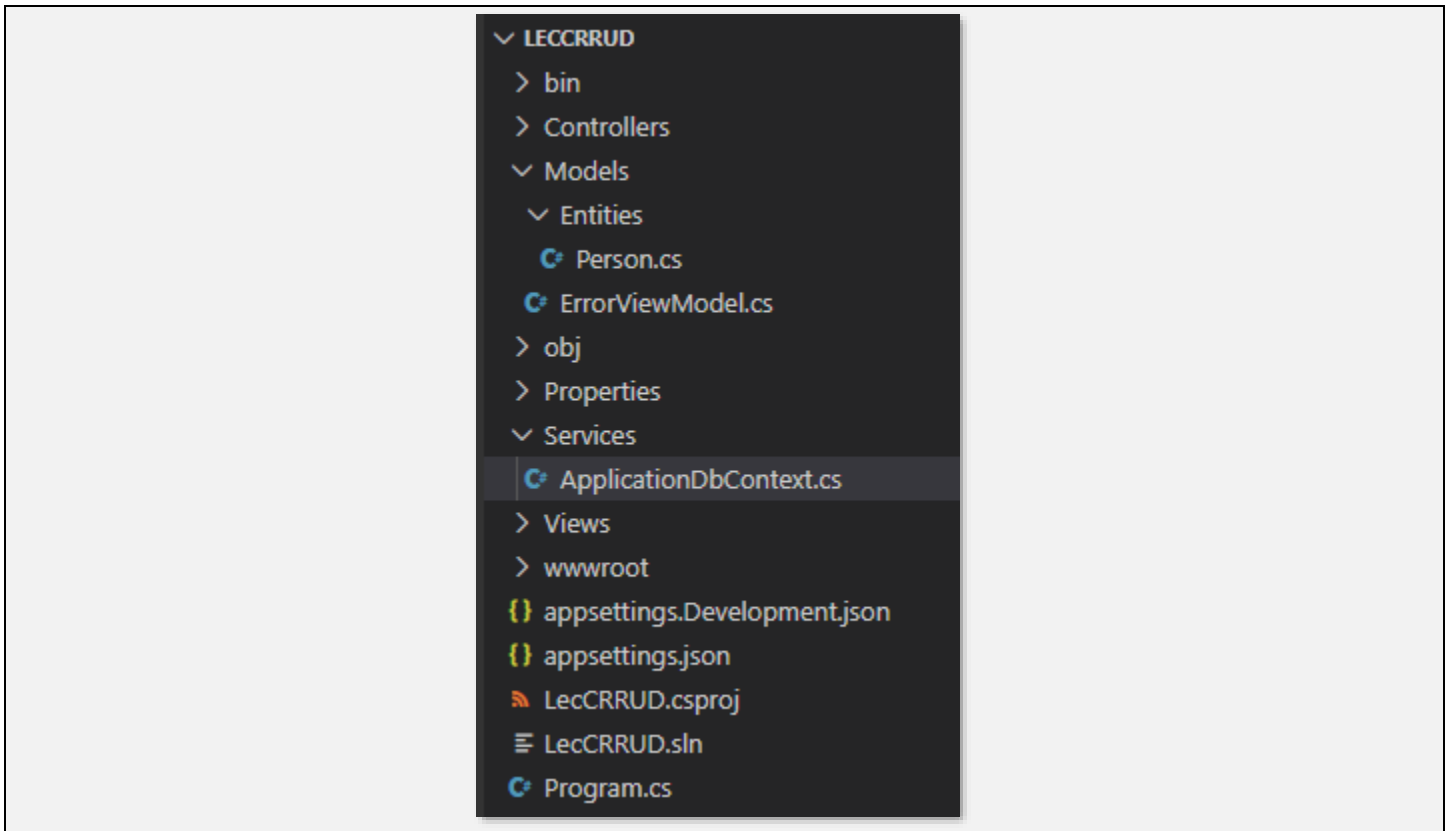
public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions options) : base(options)
    {
    }

    public DbSet<Person> People => Set<Person>();
}
```

Alternative syntax:

```
public class ApplicationDbContext(DbContextOptions options)
    : DbContext(options)
{
    public DbSet<Person> People => Set<Person>();
}
```

- ❖ We use the DbContextOptions parameter in the constructor to link the DbContext to the database through the **connection string** (see below).
- ❖ Each DbSet represents the associated table in the database.
 - Entity Framework will use the property names to name the generated tables.
 - Set<Type>() returns a DbSet<Type> instance for access to entities of the given type
- ❖ **Recommendation:** Create the DbContext class in a *Services* folder



The Connection String

- ❖ You will need to set up the database connection string.
- ❖ Best practice dictates that it is stored externally of the source code
 - e.g., in the configuration file 'appsettings.json.'

appsettings.json

```
{
  TODO: Insert the connection string here,
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  },
  "AllowedHosts": "*"
}
```

- ❖ In the example below, the connection string is named "DefaultConnection" but it could be called anything.
- ❖ The connection string names the database to be created, in this case "PersonDb."
- ❖ You should create a folder called *Data* within your project

```
"ConnectionStrings": {
  "DefaultConnection": "Data Source=_change_me.db"
},
"ConnectionStrings": {
  "DefaultConnection": "Data Source=../Data/PersonDb.db"
},
```

- ❖ The connection string needs to be linked to the DbContext.
 - This is configured in *Program.cs*.
 - The UseSqlite extension method is found in the Microsoft.EntityFrameworkCore namespace.

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContext<YourDbContext>(options =>
    options.UseSqlite(
        builder.Configuration.GetConnectionString("DefaultConnection"))
);

builder.Services.AddControllersWithViews();
...
```

EFCore Migrations

- ❖ We use EFCore migration to manage the process of transforming the entity classes into the database schema.
- ❖ We need the **Microsoft.EntityFrameworkCore.Tools** package to use migrations.
 - dotnet add package Microsoft.EntityFrameworkCore.Tools --version 9.*
- ❖ We must add migrations to be able to generate and update the database from code.
- ❖ We may need to update the specific migration version:
 - dotnet tool update --global dotnet-ef --version 9.0.12

Migrations

- ❖ **Tip:** You can seed the database during migration with the following in the DbContext:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Person>().HasData(
        new Person { Id=1, FirstName = "John", LastName = "Doe", DateOfBirth =
        DateTime.Parse("1/1/1980") }
    );
}
```

- ❖ To add a migration:
 - Give the migration a name.
 - In this example, the name is 'Mig01'.

```
dotnet ef migrations add Mig1
```


- ❖ If the command is successful, a *Migrations* folder is created along with the classes needed to perform the migration.



```
└─ Migrations
   ├── 20220205135309_Mig01.cs
   ├── ApplicationDbContextModelSnapshot.cs
   └─ Models
```

Database Update

- ❖ We use the Database Update command to update the database based on the added migration.

```
dotnet ef database update
```

Asynchronous Methods

When code runs synchronously, it operates like a single-file line: the program must wait for each operation to finish before moving to the next one. Whether it's reading a file, fetching data from a database, or performing calculations, everything comes to a halt until that task completes. This blocking behavior creates problems for users, who experience frozen interfaces and unresponsive applications while they wait.

Asynchronous code solves this problem by allowing programs to juggle multiple tasks simultaneously. Instead of stopping everything to wait for a slow operation, the application can set that task running in the background and immediately move on to handle other work. This approach improves the user experience; the UI stay responsive, buttons remain clickable, and applications feel smooth and fast even when performing time-intensive operations behind the scenes.

I/O-Bound

- ❖ An operation is I/O-bound if the time it takes to be completed is determined primarily by the period spent waiting for input/output operations to be completed.
 - E.g., requesting data from a network, accessing a database, reading, and writing to a file system
- ❖ The application is blocked until the I/O operation is completed.
- ❖ If your code will be "waiting" for something, then the work is I/O-bound

CPU-Bound

- ❖ An operation is CPU-bound (or compute bound) if the time it takes to be completed is determined primarily by speed of the CPU.
 - E.g., performing a long running calculation
- ❖ The application is delayed until the computation is completed.
- ❖ If your code will be performing an expensive computation, then it is CPU-bound

Asynchronous Programming in .NET

- ❖ If the work is **I/O-bound**, use `async` and `await`.
- ❖ If the work is **CPU-bound** and responsiveness is important, use `async` and `await`, but spawn off the work on another thread with `Task.Run`.
E.g.,

```
var t = Task.Run(() => SomeLongRunningOp() );
await t.Yield();
```

async and await

- ❖ The `async` keyword turns a method into an asynchronous method, which allows the use of the `await` keyword in its body
- ❖ The `await` keyword suspends the calling method and yields control back to its caller until the awaited task is complete
 - Can only be used inside an `async` method.

```
public async Task<string> GetDataAsync()
{
    // await suspends this method and yields control
    string result = await FetchFromDatabaseAsync();
    return result;
}

private async Task<string> FetchFromDatabaseAsync()
{
    await Task.Delay(2000); // Simulates 2-second database call
    return "Data from database";
}
```

CRRUD

- ❖ CRUD is an acronym for the operations: **Create**, **Read**, **Read All**, **Update**, and **Delete**.
- ❖ These operate on data in the data store.
 - Create (or insert) a new object into a data store
 - Read an object or many objects
 - Read all objects
 - Update (or edit) an existing object
 - Delete (or remove) an existing object.
- ❖ We'll use the **Repository Pattern** to create this abstraction.

Repository CRRUD Interface

Create(Object) CreateAsync(Object)	Inserts Object into the data store and returns the new object
---------------------------------------	---

Read(id) ReadAsync(id)	Returns a single object given the object's id from the data store
ReadAll() ReadAllAsync()	Returns a collection of all the objects from the data store
Update(id, Object) UpdateAsync(id, Object)	Changes the object in the data store identified by id to Object
Delete(id) DeleteAsync(id)	Removes the object identified by id from the data store

Start the CRRUD Interface and Implementation

```
namespace LecCRRUD.Services;

public interface IPersonRepository
{
}

namespace LecCRRUD.Services;

public class DbPersonRepository : IPersonRepository
{
}
```

Service Lifetimes and Registration Options

(Smith & Addie, 2016)

Transient

- ❖ Transient lifetime services are created each time they are requested.
- ❖ This lifetime works best for lightweight, stateless services.
- ❖ A new instance of the object is provided to every controller and every service.

Scoped

- ❖ Scoped lifetime services are created once per request.
- ❖ They are the same within a request but are different across requests.

Singleton

- ❖ Singleton lifetime services are created the first time they are requested and then every subsequent request will use the same instance.
- ❖ They are the same for every object and every request.

Example: Add the IPersonRepository scoped service

```
Program.cs
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContext<ApplicationDbContext>(options => {
    options.UseSqlServer(
```

```
builder.Configuration.GetConnectionString("DefaultConnection"));
});
builder.Services.AddControllersWithViews();
builder.Services.AddScoped<IPersonRepository, DbPersonRepository>();
```

DbContext Injection

- ❖ We inject the DbContext class into the repository implementation.

Example: DbPersonRepository – ApplicationDbContext Injection

```
public class DbPersonRepository : IPersonRepository
{
    private readonly ApplicationDbContext _db;

    public DbPersonRepository(ApplicationDbContext db)
    {
        _db = db;
    }
}
```

Alternative syntax:

```
public class DbPersonRepository(ApplicationDbContext db)
    : IPersonRepository
{
    private readonly ApplicationDbContext _db = db;
}
```

Repository Injection

- ❖ After we configure the repository service, we inject the repository into controllers that need the service.

Example: PersonController – IPersonRepository Injection

```
public class PersonController : Controller
{
    private readonly IPersonRepository _personRepo;

    public PersonController(IPersonRepository personRepo)
    {
        _personRepo = personRepo;
    }

    public IActionResult Index()
    {
        return View();
    }
}
```

ReadAll

- ❖ The `ReadAllAsync` implementation reads all records from the *Person* table and returns them as a collection.

```
public interface IPersonRepository
{
    Task<ICollection<Person>> ReadAllAsync();
}

public class DbPersonRepository : IPersonRepository
{
    ...

    public async Task<ICollection<Person>> ReadAllAsync()
    {
        return await _db.People.ToListAsync();
    }
}
```

Example: PersonController Index

As an application user, I want to view a list of all people.

Trigger:

I send a GET request to `/person/index`

Flow:

1. The application responds with the list of all people

- ❖ The Index is typically a collection of objects from the domain. For our example, a collection of Persons.
- ❖ The PersonController Index:
 1. Reads the people model
 2. Sends the model to the view

```
public async Task<IActionResult> Index()
{
    var peopleModel = await _personRepo.ReadAllAsync();
    return View(peopleModel);
}
```

- ❖ You may need to install the code generation packages:
 - `dotnet tool install -g dotnet-aspnet-codegenerator --version 9.*`
 - `dotnet add package Microsoft.VisualStudio.Web.CodeGeneration.Design --version 9.*`
- ❖ Scaffold a Razor View
 - View name: **Index**
 - Template: **List**
 - Model class: **Person**

```
dotnet aspnet-codegenerator view Index List --model Person --useDefaultLayout --relativeFolderPath Views\Person
```

Index					
Create New					
Id	FirstName	MiddleName	LastName	DateOfBirth	
1	John		Doe	1/1/1980	Edit Details Delete

Create

- ❖ The `CreateAsync` repository implementation has three steps:
 1. Add the object to the database using the database context
 2. Commit the changes
 3. Return the object

```
public interface IPersonRepository
{
    Task<ICollection<Person>> ReadAllAsync();
    Task<Person> CreateAsync(Person newPerson);
}

public async Task<Person> CreateAsync(Person newPerson)
{
    await _db.People.AddAsync(newPerson);
    await _db.SaveChangesAsync();
    return newPerson;
}
```

Example: PersonController Create

As an application user, I want to create a new person.

Trigger:

I send a GET request to /person/create

Flow:

1. The application responds with the create person view.
2. I enter the new person data.
3. I save the new person data.
4. The application sends a POST request to /person/create.
5. Guard [The data is valid]
 - Exception flow:** Invalid Data Entered
 - 5.1. The application does not save to the database
 - 5.2. The application responds with the populated create person view
 - 5.3. The application shows any error messages
 - 5.4. Exit flow
6. The application saves the data to the database
7. The application redirects to /person/index

In the controller, creating a person involves the following actions:

1. Presenting a form to accept data - GET: /person/create
2. Processing the submitted form - POST: /person/create (with post data)
 - 2.1. If the model is valid
 - 2.1.1. Add it to the database
 - 2.1.2. Redirect to the index view
 - 2.2. If the model is invalid
 - 2.2.1. Return the create view

```
public IActionResult Create()
{
    return View();
}

[HttpPost]
public async Task<IActionResult> Create(Person newPerson)
{
    if (ModelState.IsValid)
    {
        await _personRepo.CreateAsync(newPerson);
        return RedirectToAction("Index");
    }
    return View(newPerson);
}
```

❖ Scaffolding the GET Create Razor View

- View name: **Create**
- Template: **Create**

➤ Model: **Person**

```
dotnet aspnet-codegenerator view Create Create --model Person --useDefaultLayout --relativeFolderPath Views\Person
```

- Remove the Id data entry from the view code (we are not allowing the user to enter the id)

```
<div class="row">
  <div class="col-md-4">
    <form asp-action="Create">
      <div asp-validation-summary="ModelOnly" class="text-danger"></div>
      <del><div class="form-group">
        <del><label asp-for="Id" class="control-label"></label>
        <del><input asp-for="Id" class="form-control" />
        <del><span asp-validation-for="Id" class="text-danger"></span>
      </del></div>
      <div class="form-group">
        <label asp-for="FirstName" class="control-label"></label>
        <input asp-for="FirstName" class="form-control" />
        <span asp-validation-for="FirstName" class="text-danger"></span>
      </div>
      ...
    </form>
  </div>
</div>
```

Create

Person

FirstName

MiddleName

LastName

DateOfBirth



[Back to List](#)

Index					
Create New					
Id	FirstName	MiddleName	LastName	DateOfBirth	
1	John		Doe	1/1/1980	Edit Details Delete
2	Jeff	Wayne	Roach	12/25/1990	Edit Details Delete

Read

- ❖ The `ReadAsync` repository implementation retrieves an object from the database.
- ❖ We will read `Persons` using their `Ids`:
 - `FindAsync(id)`
 - Returns the object found given the primary key.
 - If the object is not found, then `null` is returned.
 - `FirstOrDefaultAsync(predicate)`
 - Returns the first object found given the predicate.
 - If an object is not found, then `null` is returned.

```
public interface IPersonRepository
{
    Task<ICollection<Person>> ReadAllAsync();
    Task<Person> CreateAsync(Person newPerson);
    Task<Person?> ReadAsync(int id);
}

public async Task<Person?> ReadAsync(int id)
{
    return await _db.People.FindAsync(id);
    //return await _db.People.FirstOrDefault((p) => p.Id == id);
}
```

Example: PersonController Details

As an application user, I want to view a person's information.

Trigger:

I send a GET request to `/person/details/{id}`

Flow:

1. **Guard:** [Id exists]
 - Exception flow:** Id does not exist
 - 1.1. The application redirects to `/person/index`
2. The application responds with the person details

We update the person index view code to create the **Details** hyperlink:

```
<table class="table">
  <thead>
```

```

        ...
    </thead>
    <tbody>
    @foreach (var item in Model) {
        <tr>
            ...
            <td>
                @Html.ActionLink("Edit", "Edit", new { /* id=item.PrimaryKey */ }) |
                <a asp-action="Details" asp-route-id="@item.Id">Details</a> |
                @Html.ActionLink("Delete", "Delete", new { /* id=item.PrimaryKey */ })
            </td>
        </tr>
    }
    </tbody>
</table>

```

❖ **PersonController Details(int id) → GET: /person/details/{id number}**

1. Read the person from the database.
2. If the person is not found
 - 2.1. Make a GET request to Index.
3. If the person is found
 - 3.1. Make a GET request to Details.

```

public async Task<IActionResult> Details(int id)
{
    var person = await _personRepo.ReadAsync(id);
    if (person == null)
    {
        return RedirectToAction("Index");
    }
    return View(person);
}

```

❖ **Scaffolding the Details Razor View**

- View name: **Details**
- Template: **Details**
- Model: **Person**

```

dotnet aspnet-codegenerator view Details Details --model Person --useDefaultLayout --
relativeFolderPath Views\Person

```

Details

Person

Id	1
FirstName	Jeff
MiddleName	Wayne
LastName	Roach
DateOfBirth	12/25/1990

[Edit](#) | [Back to List](#)

Update

The Update repository implementation has three major steps:

1. Use the old id to read the object to update
2. Update the properties
3. Commit the changes

```
public interface IPersonRepository
{
    Task<ICollection<Person>> ReadAllAsync();
    Task<Person> CreateAsync(Person newPerson);
    Task<Person?> ReadAsync(int id);
    Task UpdateAsync(int oldId, Person person);
}

public async Task UpdateAsync(int oldId, Person person)
{
    Person? personToUpdate = await ReadAsync(oldId);
    if (personToUpdate != null)
    {
        personToUpdate.FirstName = person.FirstName;
        personToUpdate.MiddleName = person.MiddleName;
        personToUpdate.LastName = person.LastName;
        personToUpdate.DateOfBirth = person.DateOfBirth;
        await _db.SaveChangesAsync();
    }
}
```

Example: PersonController Edit

As an application user, I want to update a person's data.

Trigger:

I send a GET request to `/person/update/{id}`

Flow:

1. Guard: [id exists]
 - Exception flow:** id does not exist
 - 1.1. The application redirects to `/person/index`
2. The application responds with the update person view
3. I enter the updated data
4. I save the updated data
5. The application sends a POST request to `/person/update`
6. Guard: [The data is valid]
 - Exception flow:** The data is invalid.
 - 6.1. The application does not save to the database
 - 6.2. The application responds with the update person view
 - 6.3. The application shows any error messages
 - 6.4. Exit flow
7. The application saves the data to the database
8. The application redirects to `/person/index`

We update the edit link In the Person Index Razor View and the details view:

```
<a asp-action="Edit" asp-route-id="@item.Id">Edit</a>
```

```
<a asp-action="Edit" asp-route-id="@Model.Id">Edit</a>
```

The steps are:

1. Present a form with the values filled in.
2. Process the submitted form.

`PersonController Edit(int id)` maps to GET: `/person/edit/{id number}`

1. Read the person from the database
2. If no person was found, redirect to the person index page.

`PersonController Edit(Person person)` maps to POST: `/person/edit` (with post data)

1. If the model is valid
 - 1.1. Update the person in the database
 - 1.2. Redirect to the person index page
2. If the model is invalid
 - 2.1. Return the view with the model

```
public async Task<IActionResult> Edit(int id)
{
    var person = await _personRepo.ReadAsync(id);
    if (person == null)
```

```

    {
        return RedirectToAction("Index");
    }
    return View(person);
}

[HttpPost]
public async Task<IActionResult> Edit(Person person)
{
    if (ModelState.IsValid)
    {
        await _personRepo.UpdateAsync(person.Id, person);
        return RedirectToAction("Index");
    }
    return View(person);
}

```

❖ Scaffolding the edit razor view

- View name: **Edit**
- Template: **Edit**
- Model class: **Person**

```
dotnet aspnet-codegenerator view Edit Edit --model Person --useDefaultLayout --
relativeFolderPath Views\Person
```

- **Important:** Remove the Id data entry from the view code and replace it with a hidden input for Id:

Id	FirstName	MiddleName	LastName	DateOfBirth	
1	John		Doe	1/1/1980	Edit Details Delete
2	Jeff	Wayne	Roach	12/25/1990	Edit Details Delete

Person

FirstName

MiddleName

LastName

DateOfBirth



Save

[Back to List](#)

Person

FirstName

MiddleName

LastName

DateOfBirth



Save

[Back to List](#)

Id	FirstName	MiddleName	LastName	DateOfBirth	
1	John		Doe	1/1/1980	Edit Details Delete
2	Jeffy	Wayney	Roachy	11/26/1991	Edit Details Delete

Delete

❖ The Delete implementation has three steps:

1. Retrieve the object to delete
2. Delete the object from the database
3. Commit the changes

```
public interface IPersonRepository
{
    Task<ICollection<Person>> ReadAllAsync();
    Task<Person> CreateAsync(Person newPerson);
    Task<Person?> ReadAsync(int id);
    Task UpdateAsync(int oldId, Person person);
    Task DeleteAsync(int id);
}
```

```
public async Task DeleteAsync(int id)
{
    Person? personToDelete = await ReadAsync(id);
    if (personToDelete != null)
    {
        _db.People.Remove(personToDelete);
        await _db.SaveChangesAsync();
    }
}
```

Example: PersonController Delete

As an application user, I want to delete a person's data.

Trigger:

I send a GET request to /person/delete/{id}

Flow:

1. **Guard:** [id exists]
 - Exception flow:** [id does not exist]
 - 1.1. The application redirects to /person/index
2. The application responds with the Delete Person view
3. The application confirms that I want to delete the person's data
4. **Alt:**
 - 4.1. [I answer yes]
 - 4.1.1. The application deletes the data from the database
 - 4.2. [I answer no]
 - 4.2.1. The application does not delete the data from the database
5. The application redirects to /person/index

❖ In the Person Index Razor View, we update the delete hyperlink:

```
<a asp-action="Delete" asp-route-id="@item.Id">Delete</a>
```

- ❖ Deleting a person involves the following:
 1. Presenting the UI to confirm the delete
 2. Processing the confirmation
- ❖ `PersonController Delete(int id)` maps to GET: /person/delete/{id number}
 1. Read the person from the database
 2. If the person was not found, redirect to the person index view
 3. If the person was found, return the delete view
- ❖ `PersonController DeleteConfirmed(int id)` → POST: /person/delete (with post data)
 1. Delete the person from the database using the id
 2. Make a GET request to the index page (redirect)
 3. **Note:** the name of the action method is `DeleteConfirmed`, and we use `ActionName` to set the name to `Delete`

```

public async Task<IActionResult> Delete(int id)
{
    var person = await _personRepo.ReadAsync(id);
    if (person == null)
    {
        return RedirectToAction("Index");
    }
    return View(person);
}

[HttpPost, ActionName("Delete")]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    await _personRepo.DeleteAsync(id);
    return RedirectToAction("Index");
}

```

❖ Scaffolding the Razor View

- View name: **Delete**
- Template: **Delete**
- Model class: **Person**

```
dotnet aspnet-codegenerator view Delete Delete --model Person --useDefaultLayout --relativeFolderPath Views\Person
```

- Add the hidden input for Id in the form (see below)

```

...
<form asp-action="Delete">
    <input asp-for="Id" type="hidden" />
    <input type="submit" value="Delete" class="btn btn-danger" /> |
    <a asp-action="Index">Back to List</a>
</form>
...

```

Id	FirstName	MiddleName	LastName	DateOfBirth	
1	John		Doe	1/1/1980	Edit Details Delete
2	Jeffy	Wayney	Roachy	11/26/1991	Edit Details Delete
3	To	be	Deleted	9/6/2025	Edit Details Delete

[Delete](#)
[Back to List](#)

Page 25 of 25